Market Basket Analysis for Product-to-Product Cross Sell Recommendations

This report presents the development of a Market Basket Analysis system designed to identify product-to-product cross-sell opportunities from transactional retail data. The analysis is implemented using Apache Spark and the FP-Growth algorithm to discover frequent itemsets and generate association rules.

The objective is to identify relationships between products that are frequently purchased together and evaluate the effectiveness of these relationships for recommendation purposes.

1. Business Objective

Retailers often seek to increase basket value by recommending complementary products to customers during the purchase process. Market Basket Analysis enables the discovery of such relationships by analyzing historical transaction data.

The primary objectives of this analysis are:

- Identify products that are frequently purchased together
- Generate association rules for cross selling recommendations
- Evaluate recommendation performance using holdouts test data
- Visualize relationships between products using statistical plots

2. Methodology

The workflow implemented in the notebook follows these major steps:

1. Data Loading and Preparation
2. Transaction Basket Creation
3. Model Training using the FP-Growth Algorithm
4. Generation of Association Rules
5. Evaluation using the Hide One Item Technique
6. Visualization of Product Relationships

Apache Spark was used to process the dataset efficiently and to apply scalable frequent pattern mining algorithms.

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import collect_set, size, col, expr, array_except
from pyspark.sql.functions import explode, rand
from pyspark.sql.functions import sum as spark_sum
from pyspark.ml.fpm import FPGrowth
```

This code imports PySpark libraries and functions needed for processing large datasets and performing Market Basket Analysis using the FP-Growth algorithm. `SparkSession` initializes the Spark environment. Functions like `collect_set`, `size`, `col`, `expr`, `array_except`, `explode`, and `rand` help in data transformation, basket creation, and analysis of items in transactions. `sum` is imported as `spark_sum` for aggregation operations. Finally, `FPGrowth` is used to identify frequent itemsets and generate association rules**, which helps find products that are often bought together for cross-selling insights.

```python
# 1️⃣ Start Spark
# ---------------------------------------------------

spark = SparkSession.builder \
    .appName("MarketBasketEvaluation") \
    .getOrCreate()
```

This initializes a PySpark environment so that large-scale data processing can be performed. `SparkSession.builder` is used to create a Spark session, which is the main entry point for working with Spark DataFrames and executing distributed computations. The `.appName("MarketBasketEvaluation")` assigns a name to the Spark application for identification in logs and monitoring tools, and `.getOrCreate()` either creates a new Spark session or returns an existing one if it is already running. This step is necessary before loading data or running algorithms like FP-Growth in a Spark-based Market Basket Analysis workflow.

```python
# 2️⃣ Load Dataset
# ---------------------------------------------------

df = spark.read.csv(
    "full_file.csv",
    header=True,
    inferSchema=True
).select("order_id", "product_name")
```

This code loads a CSV dataset into a Spark DataFrame for analysis. `spark.read.csv()` reads the file **"full_file.csv", where `header=True` tells Spark that the first row contains column names and `inferSchema=True` automatically detects the data types of the columns. After loading the data, `.select("order_id", "product_name")` keeps only the order ID and product name columns, which are required for creating transaction baskets in Market Basket Analysis.

```python
# 3️⃣ Train/Test Split (Leakage Safe)
# ---------------------------------------------------

orders = df.select("order_id").distinct()

train_orders, test_orders = orders.randomSplit([0.8, 0.2], seed=42)

train_df = df.join(train_orders, "order_id")
test_df = df.join(test_orders, "order_id")
```

This code splits the dataset into training and testing sets while preventing data leakage. First, it extracts all unique order IDs using `.distinct()` so that each transaction is treated as a single basket. Then `randomSplit([0.8, 0.2], seed=42)` divides the orders into 80% training data and 20% testing data** in a reproducible way. Finally, `train_df` and `test_df` are created by joining the original dataset with the split order IDs, ensuring that all items from the same order stay together in either the training set or the test set.

```python
# 4️⃣ Create Transactions
# ---------------------------------------------------

train_transactions = train_df.groupBy("order_id") \
    .agg(collect_set("product_name").alias("items"))

test_transactions = test_df.groupBy("order_id") \
    .agg(collect_set("product_name").alias("items"))
```

This converts the order-level data into transaction baskets for both training and testing datasets. It groups the data by `order_id` and uses `collect_set("product_name")` to combine all products purchased in the same order into a single array called `items`, while removing duplicates. The result is that each row represents one transaction with a list of items bought together, which is the required input format for algorithms like FP-Growth used in Market Basket Analysis.

```python
# 5️⃣ Train FP-Growth Model
# ---------------------------------------------------

fp = FPGrowth(
    itemsCol="items",
    minSupport=0.005,
    minConfidence=0.35
)

model = fp.fit(train_transactions)

print("Top Association Rules")
model.associationRules.orderBy("confidence", ascending=False).show(10, False)
```

This code trains an FP-Growth model to perform Market Basket Analysis on the training transactions. The `FPGrowth` algorithm is initialized with `itemsCol="items"` to specify the column containing the basket of products, `**minSupport=0.005**` to find item combinations that appear in at least 0.5% of transactions, and `**minConfidence=0.35**` to generate association rules with at least 35% reliability. The model is then trained using `fp.fit(train_transactions)`. Finally, the code prints the top 10 association rules sorted by highest confidence using `model.associationRules.orderBy("confidence", ascending=False).show(10, False)`, helping identify products that are frequently purchased together.

Results

```
Top Association Rules
+--------------------+----------+------------------+------------------+--------------------+
|antecedent          |consequent|confidence        |lift              |support             |
+--------------------+----------+------------------+------------------+--------------------+
|[Organic Fuji Apple]|[Banana]  |0.3733206564424786|2.5444583411332222|0.010259783489144352|
|[Honeycrisp Apple]  |[Banana]  |0.3575757575757576|2.4371451277860254|0.0089817127964982  |
+--------------------+----------+------------------+------------------+--------------------+
```

```
# 6 Hide-One-Item Evaluation
# ---------------------------------------------------

# explode items
exploded = test_transactions.select(
    "order_id",
    explode("items").alias("hidden_item"),
    "items"
)

# randomly choose one item per basket
hidden_items = exploded.withColumn(
    "rand", rand()
).groupBy("order_id").agg(
    expr("min_by(hidden_item, rand)").alias("hidden_item"),
    expr("first(items)").alias("items")
)

# create input basket (remove hidden item)
test_input = hidden_items.withColumn(
    "input_items",
    array_except(col("items"), expr("array(hidden_item)"))
)
```

This prepares the test dataset for hide-one-item evaluation to measure how well the recommendation model works. First, `explode("items")` creates separate rows for each product in a basket. Then a random item from each order is selected as the `hidden_item` using `rand()` and `min_by`, simulating a product that the model needs to predict. Finally, `array_except` removes this hidden item from the original basket to create `input_items`, which represents the partial basket given to the model, while the hidden item acts as the actual item used later to evaluate prediction accuracy.

```python
# 7️⃣ Generate Predictions
# ------------------------------------------------

prediction_input = test_input.select(
    col("order_id"),
    col("input_items").alias("items"),
    col("hidden_item")
)

predictions = model.transform(prediction_input)
```

This code generates product recommendations using the trained FP-Growth model. First, `prediction_input` selects the required columns: `order_id`, the partial basket (`input_items`) renamed as `items`, and the `hidden_item` used for evaluation. Then `model.transform(prediction_input)` applies the trained FP-Growth model to these baskets to produce predicted items (recommendations) based on the association rules learned during training.

```python
# 8️⃣ Check if hidden item is recommended
# ------------------------------------------------

evaluation = predictions.withColumn(
    "hit",
    expr("array_contains(prediction, hidden_item)")
)
```

This evaluates whether the model correctly predicted the hidden item in the recommendation results. It creates a new column called `hit` in the `predictions` DataFrame using `withColumn`, where `array_contains(prediction, hidden_item)` checks if the hidden item (the actual product removed from the basket) appears in the model's predicted recommendations. If the hidden item is present in the predicted list, `hit` is marked true (successful recommendation); otherwise, it is false.

```python
metrics = evaluation.select(
    spark_sum(expr("case when hit then 1 else 0 end")).alias("hits"),
    spark_sum(size("prediction")).alias("total_predictions"),
    expr("count(*)").alias("total_baskets")
)

values = metrics.collect()[0]

hits = values["hits"]
total_predictions = values["total_predictions"]
total_baskets = values["total_baskets"]

hit_rate = hits / total_baskets if total_baskets > 0 else 0
precision = hits / total_predictions if total_predictions > 0 else 0
recall = hits / total_baskets if total_baskets > 0 else 0

print("------------ Evaluation ------------")
print("Total Baskets:", total_baskets)
print("Hits:", hits)
print("Hit Rate:", hit_rate)
print("Precision:", precision)
print("Recall:", recall)
```

This code calculates evaluation metrics to measure the performance of the recommendation model. It first aggregates results from the `evaluation` DataFrame to compute the number of correct recommendations (`hits`), the total number of predicted items (`total_predictions`), and the total number of baskets evaluated (`total_baskets`). These values are collected from Spark into Python variables, after which the code calculates **Hit Rate, Precision, and Recall** using simple formulas. Finally, it prints these metrics, providing a summary of how accurately the FP-Growth model recommends the hidden items in the test baskets.

Results-

```
...  ------------ Evaluation ------------
     Total Baskets: 220237
     Hits: 334
     Hit Rate: 0.001516548082293166
     Precision: 0.04880187025131502
     Recall: 0.001516548082293166
```

```python
from pyspark.sql import functions as F

avg_items_per_order = (
    df.groupBy("order_id")
      .agg(F.count("product_name").alias("items_per_order"))
      .agg(F.avg("items_per_order").alias("avg_items_per_order"))
)

avg_value = avg_items_per_order.first()["avg_items_per_order"]
print(avg_value)
```

This code calculates the average number of products purchased per order in the dataset. It first groups the data by `order_id` and counts how many `product_name` entries exist in each order, creating a column called `items_per_order`. Then it calculates the average of these counts using `avg()` to determine the average number of items per order. The result is extracted using `.first()` and printed, showing the **average basket size across all transactions.

Results

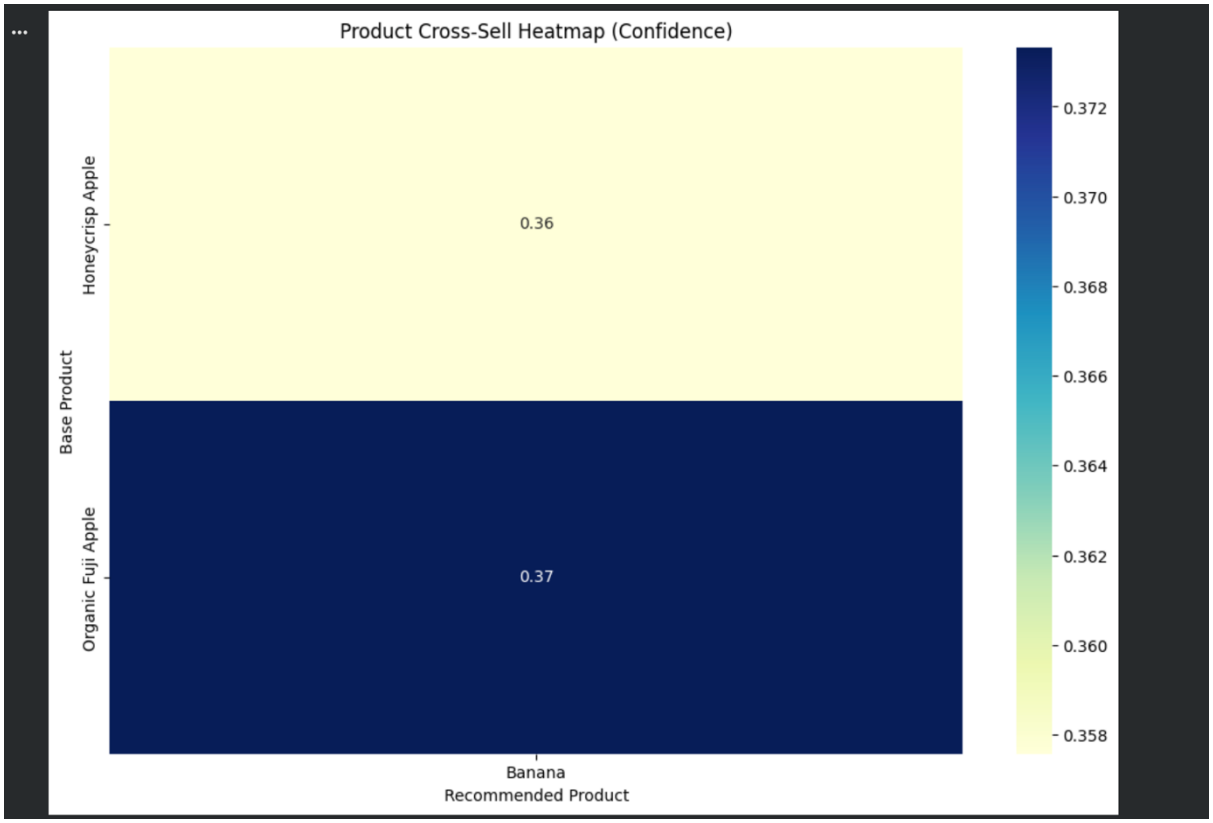
```
•••  10.139386820093375
```

Visualization

```python
# -------- Product Cross-Sell Heatmap --------

import seaborn as sns
import matplotlib.pyplot as plt

# Get association rules from FP-Growth model
rules = model.associationRules

# Explode arrays so each rule becomes a row
rules_heatmap = rules.selectExpr(
    "explode(antecedent) as product_A",
    "explode(consequent) as product_B",
    "confidence"
)

# Take top rules to keep visualization readable
top_rules = rules_heatmap.orderBy("confidence", ascending=False).limit(30)

# Convert small result to pandas for plotting
heatmap_df = top_rules.toPandas()

# Create matrix for heatmap
heatmap_matrix = heatmap_df.pivot(
    index="product_A",
    columns="product_B",
    values="confidence"
)

# Plot heatmap
plt.figure(figsize=(12,8))
sns.heatmap(heatmap_matrix, annot=True, cmap="YlGnBu")
plt.title("Product Cross-Sell Heatmap (Confidence)")
plt.xlabel("Recommended Product")
plt.ylabel("Base Product")
plt.show()
```

This creates a **product cross-sell heatmap** visualization from the FP-Growth association rules. It first extracts the association rules from the trained model and uses `explode` to convert the `antecedent` and `consequent` arrays into individual rows representing product pairs (`product_A → product_B`) along with their confidence values. The top 30 rules with the highest confidence are selected to keep the visualization readable, then converted to a Pandas DataFrame. A pivot table is created to form a matrix where rows represent the base product, columns represent the recommended product, and the values represent confidence scores. Finally, Seaborn and Matplotlib plot a heatmap, visually showing which products are strongly associated and likely to be cross-sold together.

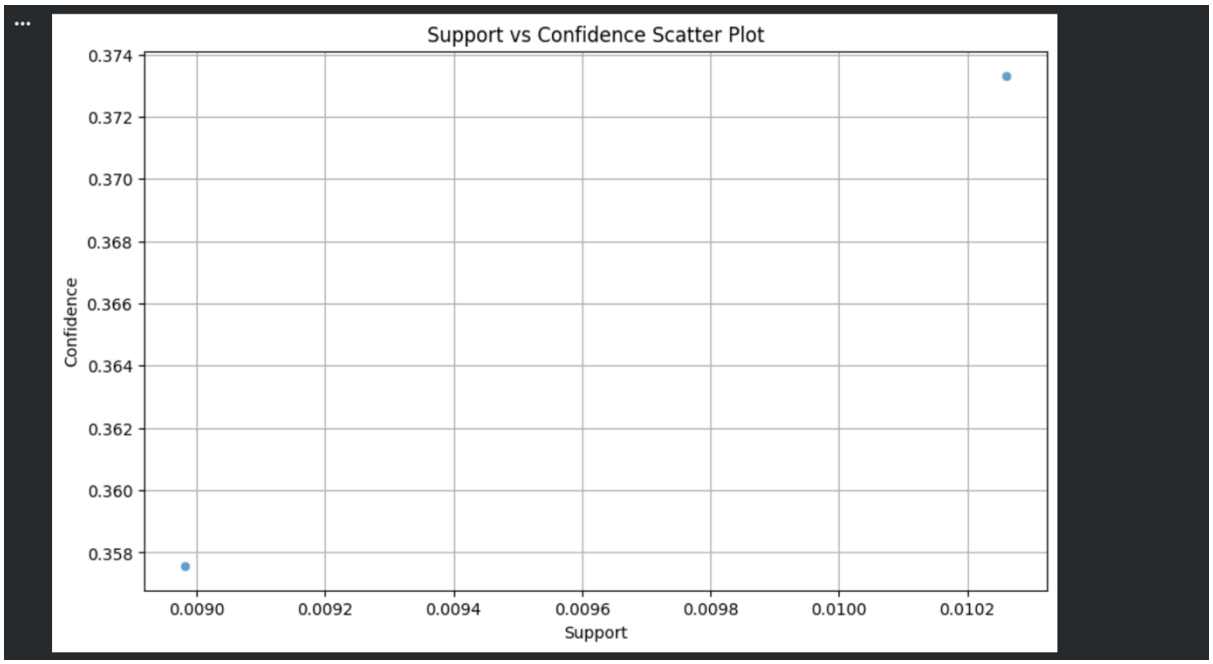
Results –




```
+import matplotlib.pyplot as plt
+import seaborn as sns
+
+# Get association rules from FP-Growth model
+rules_df = model.associationRules.select("support", "confidence").toPan
+
+# Create the scatter plot
+plt.figure(figsize=(10, 6))
+sns.scatterplot(data=rules_df, x="support", y="confidence", alpha=0.7)
+plt.title("Support vs Confidence Scatter Plot")
+plt.xlabel("Support")
+plt.ylabel("Confidence")
+plt.grid(True)
+plt.show()
```

This code creates a **Scatter plot** to visualize the relationship between support and confidence of association rules generated by the FP-Growth model. It first extracts the `support` and `confidence` values from the model's association rules and converts them into a Pandas DataFrame. Then, using Seaborn and Matplotlib, it plots a scatter graph where the x-axis represents support (how frequently an itemset appears in transactions) and the y-axis represents confidence (how reliable the rule is). This visualization helps analyze the strength and frequency of product associations in Market Basket Analysis.

Results-




```
spark.stop()
```

This is the final code which stops the active Spark session and releases all resources used by Spark. Calling `spark.stop()` properly shuts down the Spark application, freeing memory and terminating background processes associated with the Spark environment once the data processing or analysis is complete.

Model Evaluation

To assess recommendation performance, a Hide One Item evaluation strategy was used. For each transaction in the test dataset, one item is randomly removed and treated as the ground truth item.

The model then generates recommendations using the remaining items in the basket. If the hidden item appears within the recommended list, the prediction is counted as a successful hit.

Key metrics used for evaluation include:

- Hit Rate – proportion of baskets where the hidden item was successfully predicted
- Precision – proportion of recommended items that were correct
- Recall – ability of the model to recover the hidden item from the basket

These metrics provide an indication of how effective the association rules are for real■world product recommendation scenarios.

Visualization and Insights

Two primary visualizations were produced during the analysis:

1. Product Cross Sell Heatmap
Displays the confidence levels of association rules between pairs of products. Higher confidence values indicate stronger relationships between products.

2. Support vs. Confidence Scatter Plot
Illustrates the distribution of association rules and highlights rules that exhibit both strong support and high confidence.

These visualizations assist in identifying high value cross selling opportunities.

Conclusion

The Market Basket Analysis conducted in this study demonstrates the ability of frequent pattern mining techniques to uncover meaningful relationships between products in retail transaction data.

The FP-Growth algorithm efficiently identifies frequent itemsets and generates association rules that can support product recommendation strategies. When integrated into retail systems, these insights can improve cross selling, increase average order value, and enhance customer purchasing experiences.